

Reviewer Pack — Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry bdf58ab07b35 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 11:32:03 UTC

Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry ID: bdf58ab07b35 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 11:32:03 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone DNF Formulas

Statement:

For any monotone DNF formula with n variables and m clauses, the rank of the matroid formed by its clause incidence matrix is $O(\log n)$ if $m = \text{poly}(n)$, but $\Omega(n)$ for the k-CLIQUE indicator formula with $k \geq 2\sqrt{n}$.

Rationale (proposer's reasoning):

Matroid rank captures combinatorial dependencies between clauses. For sparse DNFs, the rank grows logarithmically due to union-closed set properties, but k-CLIQUE requires exponentially many clauses to represent its clique structure, forcing linear rank growth.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b7961e1b87fbe51f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]

    rank = sum(1 for row in matrix if any(row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    alpha_values = [0.5, 1, 1.5]
    results = []

    for alpha in alpha_values:
        m = int(n ** alpha)
        matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]

        rank = gaussian_elimination(matrix)
        results.append((alpha, m, rank))

    # Check k-CLIQUE instance
    k = math.ceil(2 * math.sqrt(n))
    clique_matrix = [[1 if i & (1 << j) else 0 for j in range(k)] for i in range(1 << k)]
    k_clique_rank = gaussian_elimination(clique_matrix)

    # Analyze results
    sparse_ranks = [rank for alpha, m, rank in results if alpha < 1]
    dense_rank = k_clique_rank
```

```

conjecture_holds = all(rank <= math.log(n) for rank in sparse_ranks) and dense_rank >= n
counterexample = "" if conjecture_holds else "k-CLIQUE instance does not meet the expected ran

return {
    "metric_name": "Matroid Rank",
    "metric_value": (sum(sparse_ranks) + dense_rank) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99efd595.py", line 77, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99efd595.py", line 48, in run_trial
    rank = gaussian_elimination(matrix)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99efd595.py", line 30, in gaussian_elimination
    factor = matrix[j][i] / matrix[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division by zero, preventing data collection to assess support fraction or counterexamples. | next: Debug the Gaussian elimination implementation to handle singular matrices, then re-run with diverse seeds

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110868
2	propose	ollama_remote	qwen3:8b	0	0	46896
3	preregistration	ollama_remote	qwen3:8b	0	0	24182
4	novelty	ollama_remote	qwen3:8b	0	0	21442
5	novelty	ollama_remote	qwen3:8b	0	0	15550
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12528
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8421
8	judge	ollama_remote	qwen3:8b	0	0	16984

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 256871 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/bdf58ab07b35.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bdf58ab07b35.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bdf58ab07b35.tar.gz (if generated)